Hyunsik Jeon

Global AI Strategist

Portfolio: https://portfolio-app-5ca9.vercel.app/

jeonhs9110@gmail.com | Seoul, Korea

+82 10-4092-0628 | www.linkedin.com/in/jeonhyunsik | github.com/jeonhs9110

FEATURED PROJECT 01

AIIM — AI Influencer Match

Hyunsik Jeon (Solo)

Python · FastAPI · Next.js 14 · LangGraph · GraphRAG · PostgreSQL (Cloud SQL) · Selenium · YouTube Data API · Meta Graph API · LLM · GCP Cloud Run

An end-to-end cloud platform that ingests 5,300+ creators across YouTube + Instagram + TikTok in 14 languages and **ranks brand-fit via a LangGraph agentic workflow layered on GraphRAG retrieval** — returning shortlists with per-pick rationale. Multilingual hashtag discovery surfaced 400 new YouTube creators in one 30-min run; extracted 323 IG business emails. Runs 24/7 on Google Cloud (Cloud Run + Cloud SQL). **Live:** aiim-frontend-559416574965.asia-northeast3.run.app · **GitHub:** github.com/jeonhs9110/AIIM-Showcase

Technical Pipeline Detail

1. LangGraph agentic workflow Brief parsing → candidate retrieval → multi-criteria scoring → rationale writing, modelled as a state graph (with retries, branching, human-in-the-loop hooks)

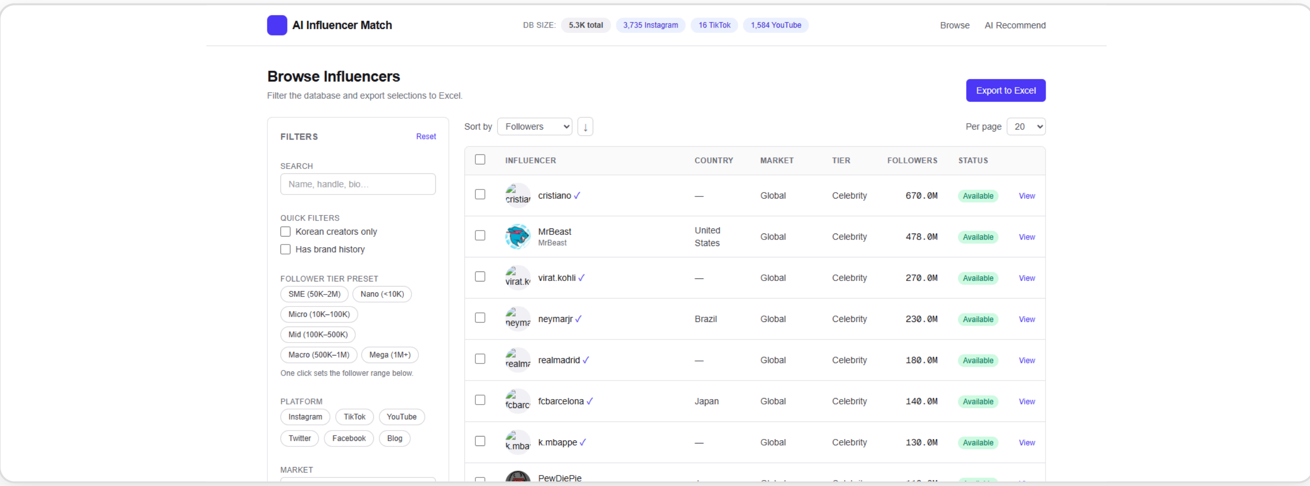
2. GraphRAG retrieval Creator · category · brand-collab relations encoded as a knowledge graph so every pick comes with a structural "why this creator" rationale

3. Multi-platform ingestion YouTube (1,584) · Instagram (3,735) · TikTok (16) — 5,300+ creators total

4. Multilingual hashtag strategy Searches 14 writing systems (광고 · スポンサー · رعاية ...) to escape English-query convergence

5. LLM enrichment Country · language · brand-collab history · vertical · contacts — auto-extracted with prompt caching

6. Infra GCP asia-northeast3 — Cloud Run + Cloud SQL Postgres + Cloud Build CI



FEATURED PROJECT 02

KOKKOK Garden — K-Beauty Multi-region E-commerce Hyunsik Jeon (Solo)

Next.js 16 · React 19 · TypeScript · Tailwind CSS 4 · AWS EC2 (t4g.small ARM64) · ALB · RDS PostgreSQL 16 · AWS Cognito · CloudFront · ACM · CloudTrail · Terraform IaC

A sustainable e-commerce designed for an administrator to keep running long after the initial build, fully owned and operated on AWS. Migrated from a Supabase-backed V1 to an AWS-native stack — EC2 + ALB + RDS PostgreSQL + Cognito + CloudFront — all provisioned with Terraform, with VPC, security groups, ACM cert provisioning, and CloudTrail audit posture personally owned. Six core data entities (products · orders · cart · users · media_stories · YouTube Shorts) live on RDS under RLS and relational constraints; every entity has a unified `/admin` CRUD + image-upload + state flow so a non-developer can operate the store. KR (purchasable) / GL (viewing) routing and 5-language product translation all run from one codebase. **Live:** ALB endpoint · **GitHub:** github.com/jeonhs9110/kok_v2

System Engineering Pipeline

1. AWS-native infrastructure Full stack on AWS: EC2 (ARM64) behind ALB, RDS PostgreSQL 16 in private subnets, CloudFront in front of static assets, ACM-issued wildcard cert, CloudTrail audit trail

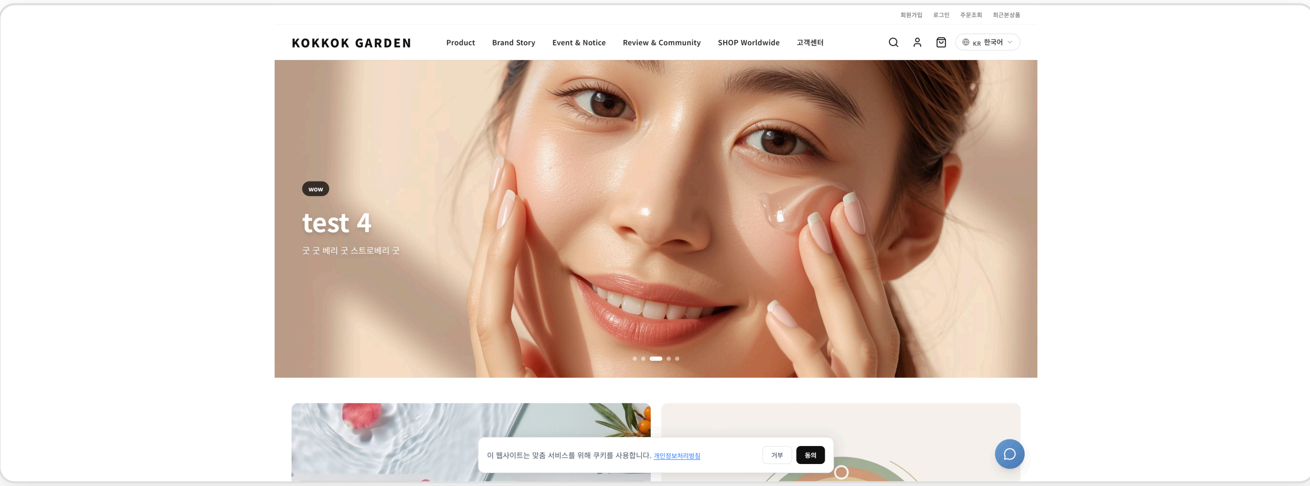
2. Terraform IaC Every resource (VPC, subnets, security groups, IAM, ALB, RDS subnet group, parameter group, Cognito) declared in Terraform; reproducible across environments

3. Sustainable admin tooling `/admin` dashboard — product · Shorts · media CRUD + image upload, operable without a developer

4. Normalised RDS schema Six entities on RDS PostgreSQL — users · products · orders · cart_items · media_stories · shorts — with RLS and relational constraints

5. AWS Cognito auth JWT verified at the Next.js middleware layer; legacy cookie path blocked

6. Multi-region routing KR/GL split via `x-user-country`, six language routes; auto-translation with 24h cache



FEATURED PROJECT 03

IAMX6 — Naver Blog Automation Suite

Hyunsik Jeon (Solo · source private)

Electron · Puppeteer-core · Node.js · LLM · Bytenode (V8) · Electron Fuses · Windows portable .exe

Key impact: **a viral-marketing engine for Naver blog owners**. Automates neighbor requests · comment/like · visitor replies with human-shaped behavior to pile up the inbound / dwell / interaction signals the Naver algorithm rewards — deliberately pushing posts toward **top search ranking (검색 상위 노출)**. Distributed as a single ~67MB portable .exe, three-layer hardened (control-flow-flattened JS + V8 bytecode + Electron Fuses). **GitHub:** github.com/jeonhs9110/IAMX6-showcase

> **Technical Pipeline Detail**
>
> **1. Viral-marketing impact** Accumulates inbound / dwell / interaction signals → maximises probability of **top search ranking (검색 상위 노출)** on Naver search and blog rankings
>
> **2. 3-workflow unified GUI** Neighbor-request · comment/like · per-visit reply + settings tab
>
> **3. Puppeteer automation** Drives a real Naver session — 2FA/CAPTCHA handled manually by the user, then the bot takes over
>
> **4. LLM comment generation** Post title + body → context-shaped Korean comments, 6–10 cached per run
>
> **5. Human simulation** Dwell time scaled to character count + ±25% jitter + ms-level noise + 35% "distraction" actions
>
> **6. 3-layer hardening** javascript-obfuscator → bytenode (.jsc) → Electron Fuses — blocks `asar extract`, debugger attach, and run-as-Node

FEATURED PROJECT 04

Top Award — IBM x RedHat AI Transformation Program

Vaccine Daily — AI News Intelligence Pipeline

Hyunsik Jeon (Vice-Leader) and 2 others

React · FastAPI · PostgreSQL · ChromaDB · HDBSCAN · ko-sroberta · Kiwi · KR-WordRank · GPT-4o-mini · GraphRAG · Docker · AWS EC2

Cutting-edge AI for cross-outlet reporting-tone comparison. A news-automation pipeline that systematically cuts LLM API cost by composing multiple Python libraries around GPT-4o-mini, and kills hallucinations with a Writer → Critic → Editor 3-agent system. HDBSCAN + ko-sroberta clustering eliminates duplicate LLM calls, a two-stage Kiwi morphological filter catches most obvious cases before the LLM, and a ChromaDB embedding cache prevents re-encoding. **Live:** jeonhs9110.github.io/vaccine-daily/ · **GitHub:** github.com/jeonhs9110/vaccine-daily

Technical Pipeline Detail

1. Multi-agent system Writer → Critic → Editor — Critic judges the draft against the source as PASS/FAIL for hallucination/fact-distortion, auto-retry on failure, async parallel up to 5 at a time

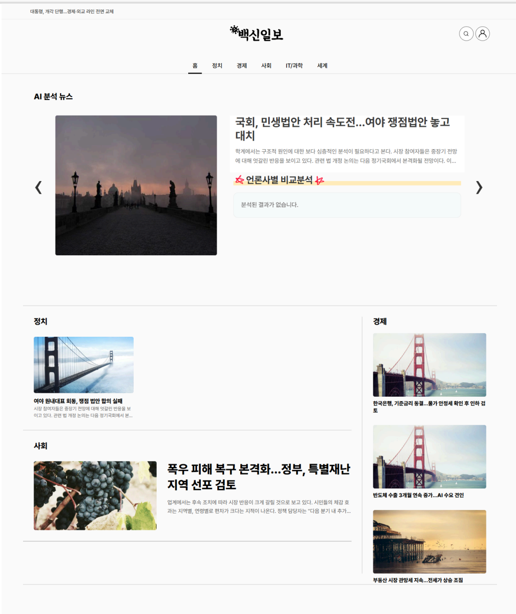
2. LLM cost reduction via multiple Python libraries HDBSCAN clustering + ko-sroberta embeddings + two-stage Kiwi morphological filter + ChromaDB embedding cache — blocks duplicate LLM calls and re-encoding

3. Data collection Selenium + BeautifulSoup crawl 10 media outlets on a 5-minute cycle

4. GraphRAG tone analysis Entity-Relation triples → knowledge graph visualising per-outlet reporting tone and framing; only the graph summary is injected into the prompt

5. Personalised recommendation Subscribed keywords + read history + category preference → multi-criteria scoring + TextRank + KR-WordRank ensemble + D3 word cloud

6. Deploy Docker Compose + Nginx + Certbot (HTTPS) + AWS EC2



FEATURED PROJECT 05

FOBO AI — European Football Prediction

Hyunsik Jeon (Solo)

Python · PyTorch · Transformer · Graph Attention Network · PPO RL · Dixon-Coles · XGBoost · LightGBM · Flask · Google Cloud

A Machine Learning and Reinforcement Learning pipeline for football-match prediction. End-to-end ML system chaining PyTorch Transformer + GAT + Dixon-Coles correction + PPO RL + XGBoost/LightGBM into an 8-stage pipeline. Across 13 European leagues: ~90% hit rate at ensemble ≥77% + favourite filter; ~95% when the PPO agent independently agrees at ≥90%. Deployed on Google Cloud via a 2-VM architecture for ~$26/month. **Live:** http://34.64.147.124/ · **GitHub:** github.com/jeonhs9110/epl-showcase

8-Step System Architecture

1. Scrape Parallel Flashscore crawl across 13 leagues (incremental re-runs)

2. Validation Integrity checks catch broken scrapes before they poison training

3. Seq optimisation Per-league look-back window (Premier ~10, Championship ~5)

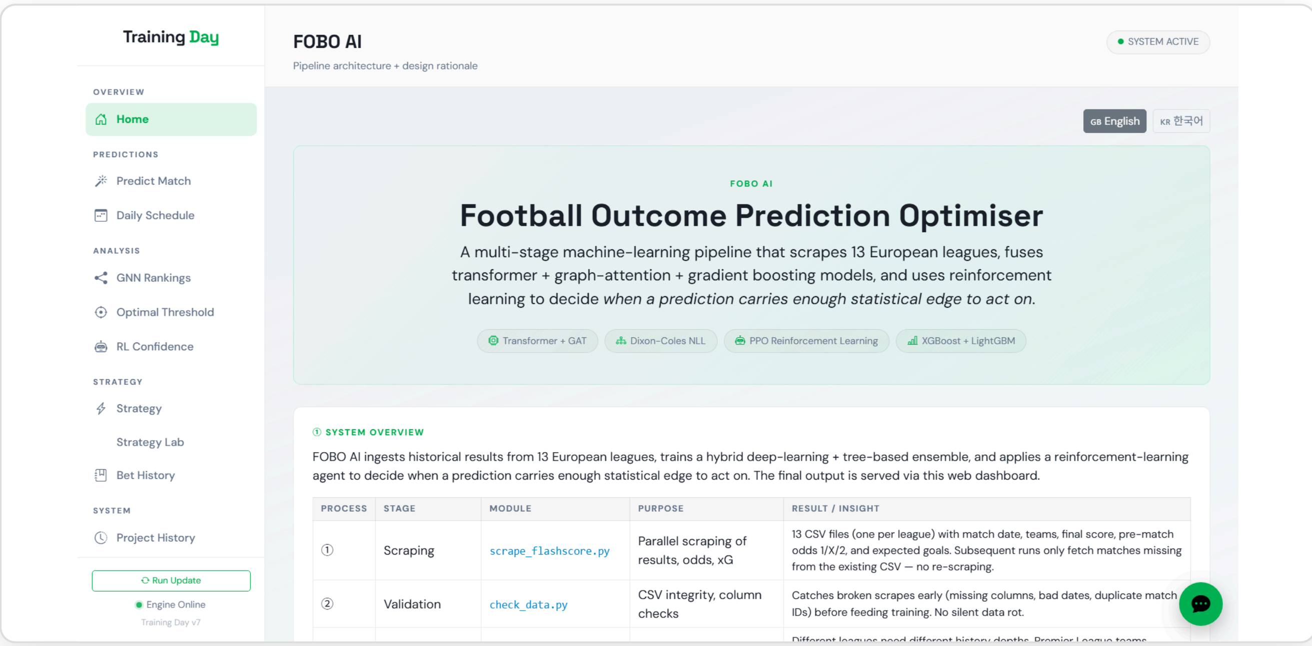
4. DL training Transformer + GAT + Dixon-Coles NLL — infers defensive posture in weak-vs-strong matchups

5. PPO RL Action ∈ {Home, Draw, Away, Pass} with Kelly-Criterion-shaped reward

6. Hybrid ensemble XGBoost + LightGBM on DL embeddings

7. Calibration Platt scaling aligns reported probability with empirical hit rate

8. Flask API Models preloaded, <200 ms per request, GPT-4o-mini chatbot scoped to today's matches



FEATURED PROJECT 06

Play_BALL — Multi-Agent Baseball Prediction Pipeline

Hyunsik Jeon (Solo)

Python · PyTorch · XGBoost · LightGBM · PyMC (Bayesian) · Knowledge Graph + vector search · faster-whisper · Streamlit · SQLite · MLB Stats API · pybaseball

A 9-agent baseball prediction pipeline that ports the FOBO AI orchestration discipline to MLB, KBO, and NPB. Hierarchical Bayesian pitcher rating → negative-binomial run model → XGBoost win/totals → Platt calibration → conformal intervals — ships predictions with confidence ranges, not single point estimates. A Knowledge Graph Reasoner extracts team-and-player entity-relation facts from Korean news, stores them as vectors, and at prediction time retrieves semantically similar past contexts to compute home/away trend signals. Bayesian model averaging across model / pundits / market / KG with fractional Kelly sizing.

> **9-Agent System Architecture**
>
> **A1 Schedule** Pulls upcoming MLB games via MLB Stats API
>
> **A2 Info Scraper** Lineups, probable pitchers, sabermetrics, park factors, weather, umpires
>
> **A3 ML Predictor** Hierarchical Bayesian pitcher rating → NB run model → XGBoost win/totals → Platt calibration → conformal intervals
>
> **A4 Pundit Aggregator** YouTube (yt-dlp + faster-whisper) / ESPN / CBS / Action Network → structured picks via local LLM
>
> **A5 Authenticator** Beta-Bernoulli reliability tracker per pundit per market
>
> **A6 Market / Odds** The Odds API → vig-adjusted implied probabilities
>
> **A7 Final Aggregator** Bayesian model averaging (ML × pundits × market × KG) + fractional Kelly
>
> **A8 Outcome Tracker** Closes the loop — updates A3 training set and A5 reliability posteriors after games settle
>
> **A9 Knowledge Graph Reasoner** Naver news → entity-relation facts → vector store; at inference, semantic search for similar past contexts yields a home/away trend signal that feeds A7 as a fourth independent input

FEATURED PROJECT 07

Synthetic Video — Local GenAI 9:16 Vertical Reel Pipeline

Hyunsik Jeon (Solo)

ComfyUI · Wan 2.2 14B GGUF · Flux schnell · PuLID · IP-Adapter · ControlNet · Florence-2 · WD14 · Depth Anything V2 · Kokoro-82M · Whisper · Stable Audio Open

Two finished 9:16 vertical reels delivered entirely on a local GPU. PuLID-Flux locks character likeness across shots, Wan 2.2 14B drives motion, Stable Audio Open generates the music bed, all composited in ComfyUI. Reel 1 (Lila venue arc): a fictional K-pop dancer ascending from a Jakarta back-alley to a stadium finale in 7 beats. Reel 2 (Mika in Tokyo 1972): a burnt-out 2026 content creator time-travels to the height of the Japanese economic boom. Each clip is 41 s @ 480 × 832, normalised to −14 LUFS streaming standard, captions baked in — ready to upload to YouTube Shorts / TikTok / Reels with no further editing.

Reusable Pipeline

1. Character lock PuLID-Flux face conditioning fixes Lila and Mika across every shot — reusable host characters for future episodes

2. Motion generation Wan 2.2 14B GGUF (HighNoise + LowNoise Q4_K_M) + Wan VAE + UMT5 drives clip-to-clip motion; Wan-Lightning LoRA speeds inference

3. Stills Flux schnell GGUF for storyboard frames; IP-Adapter + ControlNet for tight per-shot composition control

4. Captions + voice Whisper large-v3 transcribes auto-pronounced lines from Kokoro-82M TTS; captions baked into the video

5. Music Stable Audio Open generates per-reel music bed (Reel 1: EDM; Reel 2: 1970s Japanese city-pop instrumental)

6. Template Per-episode dir of CHARACTER.md + STORYBOARD.md + MUSIC.md; one script chain runs ~75–85 min unattended on a single GPU and outputs the finished mp4

FEATURED PROJECT 08

Image Finder — On-Device AI Photo Search Android App

Hyunsik Jeon (Solo)

React Native · Expo SDK 56 · YuNet (MIT) · SFace (Apache-2.0) · OpenCLIP ViT-B-32 DataComp.XL (MIT) · ONNX Runtime · AdMob (planned) · Google Play Billing (planned)

Privacy-by-default photo-search Android app. Face detection, 128-dimensional face embedding, and CLIP image+text search all run on-device — no user content leaves the handset; verified zero network egress. Personally swapped every non-commercial model (InsightFace, Apple MobileCLIP) out for an MIT / Apache-2.0 equivalent so the app is safe for an ad-monetised release. APK built; remaining work is Play Console submission, AdMob wiring, and Google Play Billing for the $3/mo ad-free tier.

Technical Pipeline Detail

1. Face detection YuNet 2023mar (MIT) replaces SCRFD / buffalo_s — ad-monetisation-safe

2. Face recognition SFace 2021dec (Apache-2.0) replaces ArcFace, producing 128-d embeddings; cluster threshold retuned to 0.33 for per-user identity grouping

3. Image+text search OpenCLIP ViT-B-32 DataComp.XL (MIT) replaces non-commercial MobileCLIP-S2 — natural-language search over the user's photo library

4. Resumable migration FACE_MODEL_VERSION + CLIP_EMBED_VERSION bumps drive automatic re-detection on next launch; existing installs migrate seamlessly

5. Privacy + permissions READ_MEDIA_IMAGES / READ_MEDIA_VIDEO only; blocks SYSTEM_ALERT_WINDOW, WRITE/READ_EXTERNAL_STORAGE — minimises Play scrutiny

6. Consent + deletion BIPA/GDPR-compliant consent flow; Settings → Data & privacy → Delete all face data lets users erase biometric identifiers on demand